

# Smart Laptop & Phone Repair Diagnosis

## Expert System Using Prolog + Tkinter UI

CM2520 - Deductive Reasoning and Logic Programming

Individual Project Blueprint, Design, Implementation Plan and Source Code

|                |                                                      |
|----------------|------------------------------------------------------|
| Prepared for   | Kavinda DC                                           |
| Project Type   | Rule-based expert system with dynamic knowledge base |
| Backend        | SWI-Prolog                                           |
| User Interface | Python Tkinter GUI                                   |
| Main Domain    | Laptop and phone repair fault diagnosis              |

This PDF is designed to be used as a starting project document and coding guide. It includes the project idea, proposal, rule design, UI plan, full Prolog backend code, full Tkinter UI code, run instructions, testing plan and presentation guidance.

# Table of Contents

1. Project Summary
2. Why This Project Is Strong and Unique
3. Mapping to DRLP Project Requirements
4. Problem Statement and Objectives
5. System Scope
6. System Architecture
7. Knowledge Base Design
8. Inference and Diagnosis Logic
9. User Interface Design
10. Dynamic Knowledge Base Features
11. Sample Diagnosis Scenarios
12. Implementation Files
13. How to Run the Project
14. Testing and Evaluation Plan
15. Presentation Plan
16. Appendix A - Prolog Source Code
17. Appendix B - Tkinter UI Source Code
18. References

# 1. Project Summary

The project is a rule-based expert system that diagnoses common laptop and phone repair issues from user-selected symptoms. A student, technician, or normal user selects symptoms such as no power, battery drain, overheating, black screen, boot failure, touch failure, charging failure, water damage, slow performance, or Wi-Fi failure. The Prolog backend reasons over the symptoms and returns possible faults, confidence score, severity, estimated cost, backup warning and repair advice.

## Core idea

Use Prolog rules to infer possible hardware/software faults from selected symptoms.

## Unique angle

Not just a symptom checker: it also gives severity, cost, backup need and repair worthiness advice.

## Main output

Possible fault, confidence score, severity, cost level, repair advice and matched symptoms.

## UI

A clean Tkinter interface with device selection, symptom checkboxes, results table and repair advice panel.

Suggested final title: **Smart Laptop & Phone Repair Diagnosis Expert System Using Prolog.**

# 2. Why This Project Is Strong and Unique

Many students may choose common Prolog projects such as bot detection, hotel reservation, employee-project matching, or crop rotation. This project is more personal, practical and demonstrable because almost everyone has experienced laptop or phone issues. It is also naturally rule-based: symptoms can be represented as facts, faults can be represented as rule conclusions, and diagnosis can be performed through logical matching.

- It solves a real everyday problem: quick first-level repair diagnosis.
- It combines hardware, software and data-safety decisions instead of only producing one answer.
- It gives a strong live demo: selecting symptoms immediately changes the diagnosis result.
- It can use dynamic knowledge base operations to add or remove new repair cases.
- It shows Prolog backtracking, lists, member/2, findall/3, assertz/1 and retractall/1 clearly.

# 3. Mapping to DRLP Project Requirements

The project guideline expects a rule-based or expert system that models real-world logic, uses facts and rules, supports dynamic knowledge base operations, includes an interface, and demonstrates built-in Prolog logic. This design directly follows those expectations.

| Requirement            | How this project satisfies it                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------|
| Facts and Rules        | Symptoms, devices, faults, severity levels, repair advice and matching rules are represented in Prolog.         |
| Dynamic Knowledge Base | The backend uses assertz/1 for observed symptoms and custom cases, and retractall/1 for clearing/removing data. |
| Interface              | The project includes a Python Tkinter GUI with checkboxes, buttons, table results and advice view.              |
| Built-in Prolog Logic  | Uses member/2, findall/3, forall/2, fail/0, dynamic predicates, list processing and rule matching.              |
| Real-world Problem     | Models a practical device repair diagnosis workflow for laptops and phones.                                     |

## 4. Problem Statement and Objectives

### Problem Statement

Users often do not know whether a device problem is caused by battery failure, display damage, charger failure, software issue, overheating, storage failure, water damage, or network fault. Repair shops can diagnose these problems, but a first-level expert system can help users understand the likely issue, urgency, data risk and repair direction before visiting a technician.

### Objectives

- 1 Build a Prolog knowledge base containing laptop and phone symptoms, faults and repair recommendations.
- 2 Develop rules that infer possible faults from selected symptoms using logical matching and scoring.
- 3 Provide severity, estimated cost and backup warning for each diagnosis.
- 4 Create a Tkinter GUI to make the system easy to use and attractive for demonstration.
- 5 Support dynamic knowledge base operations such as adding observed symptoms and adding/removing custom repair cases.
- 6 Prepare the project so it can be presented clearly with examples and test cases.

## 5. System Scope

| Included                                   | Not included / future work                                             |
|--------------------------------------------|------------------------------------------------------------------------|
| Laptop and phone fault diagnosis           | Other devices such as printers, routers and tablets can be added later |
| Hardware and software symptom categories   | No real hardware sensor reading in first version                       |
| Rule-based diagnosis with confidence score | No machine learning model in first version                             |
| Tkinter GUI and Prolog backend             | No web/mobile app in first version                                     |
| Custom case addition through UI            | Advanced database storage can be added later                           |

Recommended final project size: two device types, around 20-30 symptoms, 15-20 faults, and 10+ demonstration test cases. This is enough to be impressive but still manageable.

## 6. System Architecture

The system is divided into a Prolog reasoning backend and a Python Tkinter frontend. The UI collects selected symptoms. Python passes those symptoms to SWI-Prolog. Prolog runs the diagnosis rules and returns formatted result lines. Python parses those lines and displays a clean result table and advice panel.

|                                                                                                              |
|--------------------------------------------------------------------------------------------------------------|
| <b>Tkinter UI</b><br>Device selector, symptom checkboxes, diagnose button, result table, repair advice panel |
| DOWN                                                                                                         |
| <b>Python Controller</b><br>Builds Prolog query, calls SWI-Prolog using subprocess, parses result output     |
| DOWN                                                                                                         |

## Prolog Expert System

Facts, fault rules, symptom matching, scoring, dynamic predicates and diagnosis results

# 7. Knowledge Base Design

## Main Predicates

| Predicate          | Purpose                                                       | Example                                                           |
|--------------------|---------------------------------------------------------------|-------------------------------------------------------------------|
| device/1           | Stores supported device types                                 | device(laptop).                                                   |
| symptom/3          | Maps a device and symptom atom to a human-readable label      | symptom(laptop, overheating, "Overheating").                      |
| fault_info/7       | Stores fault label, severity, cost, advice and backup warning | fault_info(storage_failure, laptop, ...).                         |
| fault_symptoms/3   | Stores symptoms that indicate a fault                         | fault_symptoms(laptop, cooling_problem, [overheating, loud_fan]). |
| observed_symptom/1 | Dynamic symptoms selected by the user                         | observed_symptom(no_power).                                       |
| diagnose/2         | Returns ranked diagnosis results                              | diagnose(laptop, Results).                                        |

## Symptom Categories

- Power and charging: no\_power, no\_charging, battery\_drain, random\_shutdown.
- Display and touch: black\_screen, display\_flicker, cracked\_screen, touch\_not\_working.
- Performance and software: slow\_performance, boot\_failure, boot\_loop, blue\_screen, virus\_popups.
- Hardware modules: keyboard\_not\_working, camera\_not\_working, speaker\_not\_working, wifi\_not\_working.
- High-risk symptoms: water\_damage, clicking\_sound, boot\_failure and storage-related symptoms.

# 8. Inference and Diagnosis Logic

The diagnosis logic uses a matching score. Each fault has a list of required or indicative symptoms. The system compares selected symptoms with each fault's symptom list. A fault is considered likely when at least 50 percent of its symptoms are matched. Results are sorted from highest score to lowest score.

```
matched_symptoms(Required, Matched) :-
    findall(S, (member(S, Required), observed_symptom(S)), Matched).

score_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    fault_symptoms(Device, Fault, Required),
    matched_symptoms(Required, Matched),
    length(Matched, MatchCount),
    length(Required, Total),
    MatchCount > 0,
    Score is (MatchCount * 100) // Total.

likely_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    score_fault(Device, Fault, Score, MatchCount, Total, Matched),
    Score >= 50.
```

This scoring style is easier to demonstrate than a single yes/no rule because the output can show multiple possible faults and explain why each fault was suggested.

# 9. User Interface Design

The Tkinter UI should look like a small service-center dashboard. It should use a card-style layout: device and symptom selection on the left, diagnosis result table and repair advice on the right.

```
+-----+
| Smart Laptop & Phone Repair Diagnosis |
| Rule-based Prolog expert system with Tkinter interface |
+-----+
| 1. Select Device | Diagnosis Results |
| ( ) Laptop ( ) Phone | Score | Fault | Severity | Cost |
| | 100% | Battery Failure |
| 2. Select Symptoms | 75% | Charger Port Issue |
| [ ] Does not power on | Repair Advice |
| [ ] Does not charge | Fault: Battery Failure |
| [ ] Battery drains quickly | Severity: High |
| [ ] Random shutdown | Advice: Replace battery... |
| [ ] Overheating |
| [ Diagnose Now ] [ Clear ] | [ Add Custom Repair Case ] |
+-----+
```

## Main UI Components

- Header title and subtitle to show project identity.
- Radio buttons for selecting laptop or phone.
- Scrollable checklist of symptoms.
- Diagnose button to run the Prolog expert system.
- Results table showing score, fault, severity, cost and backup warning.
- Text area showing detailed repair advice and matched symptoms.
- Admin button to add custom repair cases.

## 10. Dynamic Knowledge Base Features

Dynamic knowledge base operations are important because they show that the system is not only a fixed collection of rules. In this design, Prolog dynamically stores selected symptoms at runtime and can also add or remove custom cases. The UI can save additional cases into a custom Prolog file for later use.

```
:- dynamic observed_symptom/1.
:- dynamic fault_info/7.
:- dynamic fault_symptoms/3.

add_observed_symptom(Symptom) :-
    \+ observed_symptom(Symptom),
    assertz(observed_symptom(Symptom)).

clear_observations :-
    retractall(observed_symptom(_)).

remove_case(Fault) :-
    retractall(fault_info(Fault, _, _, _, _, _, _)),
    retractall(fault_symptoms(_, Fault, _)).
```

In the presentation, you can say: *When the user selects symptoms, the system asserts them into the Prolog knowledge base as observed facts. Then the diagnosis rules infer possible faults. After diagnosis, the temporary symptoms can be retracted and replaced with a new case.*

## 11. Sample Diagnosis Scenarios

| Scenario               | Selected symptoms                              | Expected diagnosis                          |
|------------------------|------------------------------------------------|---------------------------------------------|
| Laptop battery issue   | battery_drain, random_shutdown, no_power       | Battery Failure, high severity, medium cost |
| Laptop cooling issue   | overheating, loud_fan, slow_performance        | Cooling System Problem                      |
| Laptop storage failure | boot_failure, clicking_sound, slow_performance | Hard Disk / SSD Failure, backup required    |

| Scenario               | Selected symptoms                               | Expected diagnosis                               |
|------------------------|-------------------------------------------------|--------------------------------------------------|
| Phone display damage   | cracked_screen, black_screen, touch_not_working | Display or Touch Panel Damage                    |
| Phone water damage     | water_damage, no_power, no_charging             | Water Damage, critical severity, backup required |
| Phone storage overload | storage_full, slow_performance                  | Storage Overload                                 |

## 12. Implementation Files

| File             | Purpose                                                                               |
|------------------|---------------------------------------------------------------------------------------|
| repair_expert.pl | Main Prolog backend containing facts, rules, dynamic predicates and diagnosis output. |
| app.py           | Tkinter UI that collects symptoms, calls Prolog and displays results.                 |
| custom_cases.pl  | Optional file generated by the UI to store new repair cases.                          |
| README.md        | Optional project explanation and run instructions.                                    |
| screenshots/     | Optional folder for screenshots used in the final presentation.                       |

```

repair_diagnosis_project/
|
|-- repair_expert.pl      # Prolog facts, rules and diagnosis engine
|-- app.py               # Python Tkinter graphical interface
|-- custom_cases.pl      # Optional saved custom cases
|-- README.md            # Setup and project explanation
`-- screenshots/         # UI screenshots for report/presentation

```

## 13. How to Run the Project

### Software Needed

- Python 3.x installed on the computer.
- SWI-Prolog installed and available from the command line as swipl.
- The files app.py and repair\_expert.pl saved in the same folder.

### Run Steps

- 1 Install SWI-Prolog from the official SWI-Prolog website.
- 2 Create a folder named repair\_diagnosis\_project.
- 3 Save repair\_expert.pl and app.py inside the folder.
- 4 Open terminal or command prompt in that folder.
- 5 Run: python app.py
- 6 Select laptop or phone, choose symptoms, and click Diagnose Now.

### Manual Prolog Test

```

swipl
?- [repair_expert].
?- run_diagnosis(laptop, [overheating, loud_fan, random_shutdown]).

Expected output line example:
RESULT|cooling_problem|75|high|medium|no|Cooling System Problem|Clean the fan...|overheating, loud_fan,
random_shutdown

```

## 14. Testing and Evaluation Plan

| Test ID | Input                                     | Expected behavior                                    |
|---------|-------------------------------------------|------------------------------------------------------|
| T1      | No symptom selected                       | UI shows warning message.                            |
| T2      | Laptop: overheating + loud_fan            | Cooling problem appears in result list.              |
| T3      | Laptop: clicking_sound + boot_failure     | Storage failure appears with backup warning.         |
| T4      | Phone: water_damage + no_power            | Water damage appears as critical.                    |
| T5      | Phone: cracked_screen + touch_not_working | Display/touch panel damage appears.                  |
| T6      | Add custom case                           | custom_cases.pl is updated and new case can be used. |
| T7      | Multiple matching faults                  | Results are ranked by confidence score.              |

### Evaluation Criteria

- Correctness: Does the rule base return suitable faults for selected symptoms?
- Explainability: Does each result show matched symptoms and repair advice?
- Prolog quality: Are facts, rules, lists, dynamic predicates and backtracking used clearly?
- UI quality: Is the interface clean, usable and easy to demonstrate?
- Extendability: Can new symptoms and cases be added without redesigning the whole system?

## 15. Presentation Plan

A good presentation should not only show code; it should show the real-world problem, why Prolog is suitable, how facts and rules are represented, how inference works, and then a live demo.

| Slide | Content                                                                  |
|-------|--------------------------------------------------------------------------|
| 1     | Title: Smart Laptop & Phone Repair Diagnosis Expert System               |
| 2     | Problem: Users cannot easily identify likely device faults from symptoms |
| 3     | Prolog solution: facts, rules, symptom matching and scoring              |
| 4     | Knowledge base: devices, symptoms, fault_info/7, fault_symptoms/3        |
| 5     | Inference logic: selected symptoms -> observed facts -> likely faults    |
| 6     | UI screenshots: symptom selection and result screen                      |
| 7     | Dynamic KB: assertz, retractall, custom cases                            |
| 8     | Demo scenario 1: laptop overheating                                      |
| 9     | Demo scenario 2: phone water damage                                      |
| 10    | Conclusion and future improvements                                       |

### Short Demo Script

1. Open the Tkinter application.
2. Select Laptop.
3. Choose Overheating, Fan is very loud and Random shutdown.
4. Click Diagnose Now.
5. Explain how the Prolog rule `fault_symptoms(laptop, cooling_problem, ...)` matched the symptoms.
6. Select Phone.
7. Choose Water damage, Does not power on and Does not charge.
8. Show the critical warning and backup/service advice.

## 16. Appendix A - Full Prolog Source Code

Save this file as **repair\_expert.pl**.

```
% =====
% Smart Laptop & Phone Repair Diagnosis Expert System
% Course: CM2520 - Deductive Reasoning and Logic Programming
% File: repair_expert.pl
% Run with SWI-Prolog.
% =====

:- dynamic observed_symptom/1.
:- dynamic fault_info/7.
:- dynamic fault_symptoms/3.
:- multifile fault_info/7.
:- multifile fault_symptoms/3.

% Optional custom repair cases saved from the Python UI.
:- initialization(load_custom_cases).
load_custom_cases :-
    exists_file('custom_cases.pl'), !,
    consult('custom_cases.pl').
load_custom_cases.

% -----
% Devices
% -----
device(laptop).
device(phone).

% -----
% Symptom dictionary
% symptom(Device, SymptomAtom, HumanReadableLabel).
% -----
symptom(laptop, no_power, "Does not power on").
symptom(laptop, no_charging, "Does not charge").
symptom(laptop, battery_drain, "Battery drains quickly").
symptom(laptop, random_shutdown, "Random shutdown").
symptom(laptop, overheating, "Overheating").
symptom(laptop, loud_fan, "Fan is very loud").
symptom(laptop, black_screen, "Black screen").
symptom(laptop, display_flicker, "Display flickering").
symptom(laptop, slow_performance, "Slow performance").
symptom(laptop, boot_failure, "Boot failure").
symptom(laptop, clicking_sound, "Clicking sound from storage").
symptom(laptop, keyboard_not_working, "Keyboard not working").
symptom(laptop, wifi_not_working, "Wi-Fi not working").
symptom(laptop, virus_popups, "Unwanted popups / suspicious apps").
symptom(laptop, blue_screen, "Blue screen / system crash").

symptom(phone, no_power, "Does not power on").
symptom(phone, no_charging, "Does not charge").
symptom(phone, battery_drain, "Battery drains quickly").
symptom(phone, random_shutdown, "Random shutdown").
symptom(phone, overheating, "Overheating").
symptom(phone, cracked_screen, "Cracked screen").
symptom(phone, black_screen, "Black screen").
symptom(phone, touch_not_working, "Touch not working").
symptom(phone, slow_performance, "Slow performance").
symptom(phone, storage_full, "Storage full").
symptom(phone, wifi_not_working, "Wi-Fi not working").
symptom(phone, camera_not_working, "Camera not working").
symptom(phone, speaker_not_working, "Speaker not working").
symptom(phone, water_damage, "Water damage").
```

```

symptom(phone, boot_loop, "Phone stuck in boot loop").

% -----
% Fault information
% fault_info(FaultAtom, Device, Label, Severity, Cost, Advice, BackupNeeded).
% Severity: low | medium | high | critical
% Cost: low | medium | high
% BackupNeeded: yes | no
% -----
fault_info(battery_failure, laptop, "Battery Failure", high, medium,
    "Replace the battery. Also check adapter health and charging cycle count.", no).
fault_info(charger_or_port_issue, laptop, "Charger or Charging Port Issue", medium, low,
    "Test with another adapter. Clean or replace the charging port if needed.", no).
fault_info(cooling_problem, laptop, "Cooling System Problem", high, medium,
    "Clean the fan, replace thermal paste, and check air vents.", no).
fault_info(display_damage, laptop, "Display Cable or Panel Damage", high, high,
    "Check display cable first. If the panel is damaged, replace the screen.", no).
fault_info(storage_failure, laptop, "Hard Disk / SSD Failure", critical, high,
    "Stop using the device heavily. Backup data immediately and replace storage.", yes).
fault_info(malware_infection, laptop, "Malware or Unwanted Software", medium, low,
    "Run a trusted malware scan, remove suspicious apps, and update the OS.", yes).
fault_info(ram_or_os_crash, laptop, "RAM Issue or Operating System Crash", high, medium,
    "Run memory diagnostics. Re-seat RAM or repair/reinstall the OS.", yes).
fault_info(keyboard_fault, laptop, "Keyboard Hardware Fault", medium, medium,
    "Check keyboard connector. Replace keyboard if multiple keys fail.", no).
fault_info(wifi_adapter_fault, laptop, "Wi-Fi Adapter or Driver Fault", medium, low,
    "Reinstall network driver. If still failing, test or replace Wi-Fi adapter.", no).

fault_info(phone_battery_failure, phone, "Phone Battery Failure", high, medium,
    "Replace battery. Check charging IC if battery replacement does not solve it.", no).
fault_info(phone_charging_port_issue, phone, "Charging Port Issue", medium, low,
    "Clean charging port carefully. Replace port if cable does not fit firmly.", no).
fault_info(phone_display_damage, phone, "Display or Touch Panel Damage", high, high,
    "Replace display assembly if screen/touch is damaged.", no).
fault_info(phone_storage_overload, phone, "Storage Overload", medium, low,
    "Delete unnecessary files, clear app cache, and move media to cloud/PC.", yes).
fault_info(phone_water_damage, phone, "Water Damage", critical, high,
    "Power off immediately. Do not charge. Take to service center for board cleaning.", yes).
fault_info(phone_camera_fault, phone, "Camera Module Fault", medium, medium,
    "Check camera permission and app first. Replace camera module if hardware issue remains.", no).
fault_info(phone_speaker_fault, phone, "Speaker or Audio IC Fault", medium, medium,
    "Clean speaker grill. If sound is absent, test and replace speaker module.", no).
fault_info(phone_network_fault, phone, "Network / Wi-Fi Fault", medium, low,
    "Reset network settings, update software, and check antenna or Wi-Fi module.", no).
fault_info(phone_boot_loop_fault, phone, "Boot Loop / Firmware Issue", critical, medium,
    "Backup if possible, then repair firmware. Avoid factory reset before data backup.", yes).

% -----
% Diagnostic rules
% fault_symptoms(Device, FaultAtom, RequiredSymptoms).
% A fault is likely when enough required symptoms are observed.
% -----
fault_symptoms(laptop, battery_failure,
    [battery_drain, random_shutdown, no_power]).
fault_symptoms(laptop, charger_or_port_issue,
    [no_charging, no_power]).
fault_symptoms(laptop, cooling_problem,
    [overheating, loud_fan, random_shutdown, slow_performance]).
fault_symptoms(laptop, display_damage,
    [black_screen, display_flicker]).

```

```

fault_symptoms(laptop, storage_failure,
    [boot_failure, clicking_sound, slow_performance]).
fault_symptoms(laptop, malware_infection,
    [slow_performance, virus_popups]).
fault_symptoms(laptop, ram_or_os_crash,
    [blue_screen, random_shutdown, boot_failure]).
fault_symptoms(laptop, keyboard_fault,
    [keyboard_not_working]).
fault_symptoms(laptop, wifi_adapter_fault,
    [wifi_not_working]).

fault_symptoms(phone, phone_battery_failure,
    [battery_drain, random_shutdown, no_power]).
fault_symptoms(phone, phone_charging_port_issue,
    [no_charging, no_power]).
fault_symptoms(phone, phone_display_damage,
    [cracked_screen, black_screen, touch_not_working]).
fault_symptoms(phone, phone_storage_overload,
    [storage_full, slow_performance]).
fault_symptoms(phone, phone_water_damage,
    [water_damage, no_power, no_charging]).
fault_symptoms(phone, phone_camera_fault,
    [camera_not_working]).
fault_symptoms(phone, phone_speaker_fault,
    [speaker_not_working]).
fault_symptoms(phone, phone_network_fault,
    [wifi_not_working]).
fault_symptoms(phone, phone_boot_loop_fault,
    [boot_loop, random_shutdown]).

% -----
% Dynamic observed symptoms
% -----
clear_observations :- retractall(observed_symptom(_)).

add_observed_symptom(Symptom) :-
    \+ observed_symptom(Symptom),
    assertz(observed_symptom(Symptom)).
add_observed_symptom(_).

add_symptom_list([]).
add_symptom_list([H|T]) :-
    add_observed_symptom(H),
    add_symptom_list(T).

% -----
% Matching and scoring logic
% -----
matched_symptoms(Required, Matched) :-
    findall(S, (member(S, Required), observed_symptom(S)), Matched).

score_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    fault_symptoms(Device, Fault, Required),
    matched_symptoms(Required, Matched),
    length(Matched, MatchCount),
    length(Required, Total),
    Total > 0,
    MatchCount > 0,
    Score is (MatchCount * 100) // Total.

```

```

likely_fault(Device, Fault, Score, MatchCount, Total, Matched) :-
    score_fault(Device, Fault, Score, MatchCount, Total, Matched),
    Score >= 50.

% Sort by score descending. predsor comparator receives full result rows.
compare_score(Order, result(_, ScoreA, _, _, _, _, _),
               result(_, ScoreB, _, _, _, _, _)) :-
    ( ScoreA > ScoreB -> Order = '<'
    ; ScoreA < ScoreB -> Order = '>'
    ; Order = '='
    ).

make_result(Device,
             result(Fault, Score, Label, Severity, Cost, BackupNeeded, Advice, Matched)) :-
    likely_fault(Device, Fault, Score, _MatchCount, _Total, Matched),
    fault_info(Fault, Device, Label, Severity, Cost, Advice, BackupNeeded).

diagnose(Device, ResultsSorted) :-
    findall(Result, make_result(Device, Result), Results),
    predsor(compare_score, Results, ResultsSorted).

% -----
% Console and UI output
% -----
run_diagnosis(Device, Symptoms) :-
    clear_observations,
    add_symptom_list(Symptoms),
    diagnose(Device, Results),
    print_results(Results),
    halt.

print_results([]) :-
    format('NO_RESULT|No strong fault matched|Try selecting more symptoms|~n').
print_results(Results) :-
    forall(member(R, Results), print_result_line(R)).

print_result_line(result(Fault, Score, Label, Severity, Cost, BackupNeeded, Advice, Matched)) :-
    atomic_list_concat(Matched, '|', MatchedText),
    format('RESULT|~w|~d|~w|~w|~w|~s|~s|~n',
           [Fault, Score, Severity, Cost, BackupNeeded, Label, Advice, MatchedText]).

% Backtracking demonstration: prints all known faults one by one.
list_all_faults :-
    fault_info(Fault, Device, Label, Severity, Cost, _Advice, BackupNeeded),
    format('~w | ~w | ~s | severity=~w | cost=~w | backup=~w~n',
           [Device, Fault, Label, Severity, Cost, BackupNeeded]),
    fail.
list_all_faults.

% -----
% Dynamic knowledge base administration
% -----
add_custom_case(Device, Fault, Label, Symptoms, Severity, Cost, Advice, BackupNeeded) :-
    assertz(fault_info(Fault, Device, Label, Severity, Cost, Advice, BackupNeeded)),
    assertz(fault_symptoms(Device, Fault, Symptoms)).

remove_case(Fault) :-
    retractall(fault_info(Fault, _, _, _, _, _, _)),
    retractall(fault_symptoms(_, Fault, _)).

```

```

% Example manual queries in SWI-Prolog:
% ?- [repair_expert].
% ?- clear_observations, add_symptom_list([overheating,loud_fan,random_shutdown]),
%    diagnose(laptop, Results).
% ?- run_diagnosis(phone, [battery_drain,random_shutdown,no_power]).
% ?- list_all_faults.

```

## 17. Appendix B - Full Tkinter UI Source Code

Save this file as **app.py**.

```
# =====
# Smart Laptop & Phone Repair Diagnosis Expert System
# File: app.py
# Frontend: Tkinter UI
# Backend: SWI-Prolog file repair_expert.pl
# =====

import os
import re
import subprocess
import tkinter as tk
from tkinter import ttk, messagebox

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PROLOG_FILE = os.path.join(BASE_DIR, "repair_expert.pl")
CUSTOM_FILE = os.path.join(BASE_DIR, "custom_cases.pl")

SYMPTOMS = {
    "laptop": [
        ("no_power", "Does not power on"),
        ("no_charging", "Does not charge"),
        ("battery_drain", "Battery drains quickly"),
        ("random_shutdown", "Random shutdown"),
        ("overheating", "Overheating"),
        ("loud_fan", "Fan is very loud"),
        ("black_screen", "Black screen"),
        ("display_flicker", "Display flickering"),
        ("slow_performance", "Slow performance"),
        ("boot_failure", "Boot failure"),
        ("clicking_sound", "Clicking sound from storage"),
        ("keyboard_not_working", "Keyboard not working"),
        ("wifi_not_working", "Wi-Fi not working"),
        ("virus_popups", "Unwanted popups / suspicious apps"),
        ("blue_screen", "Blue screen / system crash"),
    ],
    "phone": [
        ("no_power", "Does not power on"),
        ("no_charging", "Does not charge"),
        ("battery_drain", "Battery drains quickly"),
        ("random_shutdown", "Random shutdown"),
        ("overheating", "Overheating"),
        ("cracked_screen", "Cracked screen"),
        ("black_screen", "Black screen"),
        ("touch_not_working", "Touch not working"),
        ("slow_performance", "Slow performance"),
        ("storage_full", "Storage full"),
        ("wifi_not_working", "Wi-Fi not working"),
        ("camera_not_working", "Camera not working"),
        ("speaker_not_working", "Speaker not working"),
        ("water_damage", "Water damage"),
        ("boot_loop", "Phone stuck in boot loop"),
    ],
}

def safe_atom(text: str) -> str:
    """Convert user text to a safe Prolog atom-like identifier."""
    text = text.lower().strip()
    text = re.sub(r"[^a-z0-9_]+", "_", text)
    text = re.sub(r"_+", "_", text).strip("_")
```

```

    return text or "custom_fault"

def prolog_list(items):
    """Build a Prolog list of atoms from Python strings."""
    return "[" + ",".join(items) + "]"

def run_prolog_diagnosis(device, symptoms):
    """Call SWI-Prolog and return parsed diagnosis rows."""
    if not os.path.exists(PROLOG_FILE):
        raise FileNotFoundError("repair_expert.pl was not found in the project folder.")

    goal = f"run_diagnosis({device}, {prolog_list(symptoms)})"
    command = ["swipl", "-q", "-s", PROLOG_FILE, "-g", goal]

    completed = subprocess.run(
        command,
        capture_output=True,
        text=True,
        cwd=BASE_DIR,
        timeout=15,
        check=False,
    )

    if completed.returncode != 0:
        raise RuntimeError(completed.stderr.strip() or "SWI-Prolog failed to run.")

    rows = []
    for line in completed.stdout.splitlines():
        parts = line.split("|")
        if not parts:
            continue
        if parts[0] == "NO_RESULT":
            rows.append({
                "fault": "No strong match",
                "score": "0",
                "severity": "-",
                "cost": "-",
                "backup": "-",
                "label": parts[1] if len(parts) > 1 else "No strong fault matched",
                "advice": parts[2] if len(parts) > 2 else "Select more symptoms.",
                "matched": "",
            })
        elif parts[0] == "RESULT" and len(parts) >= 9:
            rows.append({
                "fault": parts[1],
                "score": parts[2],
                "severity": parts[3],
                "cost": parts[4],
                "backup": parts[5],
                "label": parts[6],
                "advice": parts[7],
                "matched": parts[8],
            })
    return rows

class RepairDiagnosisApp(tk.Tk):
    def __init__(self):

```

```

super().__init__()
self.title("Smart Device Repair Diagnosis Expert System")
self.geometry("1100x720")
self.minsize(950, 620)
self.configure(bg="#f4f7fb")

self.device_var = tk.StringVar(value="laptop")
self.symptom_vars = {}
self._build_style()
self._build_layout()
self.render_symptoms()

def _build_style(self):
    style = ttk.Style(self)
    style.theme_use("clam")
    style.configure("TFrame", background="#f4f7fb")
    style.configure("Card.TFrame", background="white", relief="flat")
    style.configure("Title.TLabel", font=("Segoe UI", 22, "bold"), background="#f4f7fb",
foreground="#172033")
    style.configure("Sub.TLabel", font=("Segoe UI", 11), background="#f4f7fb", foreground="#596579")
    style.configure("CardTitle.TLabel", font=("Segoe UI", 14, "bold"), background="white",
foreground="#172033")
    style.configure("TButton", font=("Segoe UI", 10, "bold"), padding=8)
    style.configure("Accent.TButton", background="#2563eb", foreground="white")
    style.configure("TCheckbutton", background="white", font=("Segoe UI", 10))
    style.configure("Treeview", font=("Segoe UI", 10), rowheight=30)
    style.configure("Treeview.Heading", font=("Segoe UI", 10, "bold"))

def _build_layout(self):
    header = ttk.Frame(self)
    header.pack(fill="x", padx=24, pady=(20, 10))
    ttk.Label(header, text="Smart Laptop & Phone Repair Diagnosis", style="Title.TLabel").pack(
anchor="w")
    ttk.Label(
        header,
        text="Rule-based Prolog expert system with a Tkinter graphical interface",
        style="Sub.TLabel",
    ).pack(anchor="w", pady=(4, 0))

    body = ttk.Frame(self)
    body.pack(fill="both", expand=True, padx=24, pady=12)

    left = ttk.Frame(body, style="Card.TFrame")
    left.pack(side="left", fill="both", expand=False, padx=(0, 12), ipadx=18, ipady=16)

    right = ttk.Frame(body, style="Card.TFrame")
    right.pack(side="right", fill="both", expand=True, padx=(12, 0), ipadx=18, ipady=16)

    ttk.Label(left, text="1. Select Device", style="CardTitle.TLabel").pack(anchor="w", padx=16,
pady=(16, 8))
    device_row = ttk.Frame(left, style="Card.TFrame")
    device_row.pack(anchor="w", padx=16, pady=(0, 16))
    ttk.Radiobutton(device_row, text="Laptop", variable=self.device_var, value="laptop", command=self.
render_symptoms).pack(side="left", padx=(0, 12))
    ttk.Radiobutton(device_row, text="Phone", variable=self.device_var, value="phone", command=self.
render_symptoms).pack(side="left")

    ttk.Label(left, text="2. Select Symptoms", style="CardTitle.TLabel").pack(anchor="w", padx=16,
pady=(0, 8))

    self.symptom_canvas = tk.Canvas(left, width=390, height=390, bg="white", highlightthickness=0)
    self.symptom_frame = ttk.Frame(self.symptom_canvas, style="Card.TFrame")
    self.symptom_scroll = ttk.Scrollbar(left, orient="vertical", command=self.symptom_canvas.yview)
    self.symptom_canvas.configure(yscrollcommand=self.symptom_scroll.set)
    self.symptom_canvas.pack(side="left", fill="both", expand=True, padx=(16, 0), pady=(0, 12))
    self.symptom_scroll.pack(side="right", fill="y", padx=(0, 16), pady=(0, 12))
    self.symptom_canvas.create_window((0, 0), window=self.symptom_frame, anchor="nw")

```

```

        self.symptom_frame.bind("<Configure>", lambda e: self.symptom_canvas.configure(scrollregion=self.
symptom_canvas.bbox("all")))

        button_row = ttk.Frame(left, style="Card.TFrame")
        button_row.pack(fill="x", padx=16, pady=(0, 16))
        ttk.Button(button_row, text="Diagnose Now", style="Accent.TButton", command=self.diagnose).pack(
side="left", fill="x", expand=True)
        ttk.Button(button_row, text="Clear", command=self.clear_selection).pack(side="left", padx=(10, 0))

        ttk.Label(right, text="Diagnosis Results", style="CardTitle.TLabel").pack(anchor="w", padx=16,
pady=(16, 8))
        columns = ("score", "fault", "severity", "cost", "backup")
        self.tree = ttk.Treeview(right, columns=columns, show="headings", height=8)
        for col, width in [("score", 70), ("fault", 240), ("severity", 100), ("cost", 90), ("backup",
110)]:
            self.tree.heading(col, text=col.title())
            self.tree.column(col, width=width, anchor="w")
            self.tree.pack(fill="x", padx=16, pady=(0, 12))
            self.tree.bind("<<TreeviewSelect>>", self.show_selected_detail)

        ttk.Label(right, text="Repair Advice", style="CardTitle.TLabel").pack(anchor="w", padx=16, pady=(
8, 8))
        self.detail = tk.Text(right, height=10, wrap="word", font=("Segoe UI", 11), bg="#f8fafc",
relief="flat")
        self.detail.pack(fill="both", expand=True, padx=16, pady=(0, 16))

        admin = ttk.Frame(right, style="Card.TFrame")
        admin.pack(fill="x", padx=16, pady=(0, 16))
        ttk.Button(admin, text="Add Custom Repair Case", command=self.add_custom_case_window).pack(
side="left")
        ttk.Button(admin, text="Open Project Folder", command=self.open_folder_hint).pack(side="left",
padx=10)

    def render_symptoms(self):
        for child in self.symptom_frame.winfo_children():
            child.destroy()
        self.symptom_vars.clear()
        device = self.device_var.get()
        for atom, label in SYMPTOMS[device]:
            var = tk.BooleanVar(value=False)
            self.symptom_vars[atom] = var
            ttk.Checkbutton(self.symptom_frame, text=label, variable=var).pack(anchor="w", pady=4, padx=4)

    def selected_symptoms(self):
        return [atom for atom, var in self.symptom_vars.items() if var.get()]

    def clear_selection(self):
        for var in self.symptom_vars.values():
            var.set(False)
        for row in self.tree.get_children():
            self.tree.delete(row)
        self.detail.delete("1.0", "end")

    def diagnose(self):
        symptoms = self.selected_symptoms()
        if not symptoms:
            messagebox.showwarning("No symptoms", "Please select at least one symptom.")
            return
        try:
            rows = run_prolog_diagnosis(self.device_var.get(), symptoms)
        except Exception as exc:
            messagebox.showerror("Prolog Error", str(exc))
            return

        for row in self.tree.get_children():
            self.tree.delete(row)
        self.results = rows
        for idx, row in enumerate(rows):

```

```

        self.tree.insert("", "end", iid=str(idx), values=(
            row["score"] + "%" if row["score"].isdigit() else row["score"],
            row["label"],
            row["severity"],
            row["cost"],
            row["backup"],
        ))
    if rows:
        self.tree.selection_set("0")
        self.show_selected_detail()

def show_selected_detail(self, event=None):
    selected = self.tree.selection()
    if not selected:
        return
    row = self.results[int(selected[0])]
    text = (
        f"Fault: {row['label']}\n"
        f"Confidence Score: {row['score']}%\n"
        f"Severity: {row['severity']}\n"
        f"Estimated Cost: {row['cost']}\n"
        f"Backup Needed: {row['backup']}\n\n"
        f"Matched Symptoms: {row['matched']}\n\n"
        f"Repair Advice:\n{row['advice']}\n"
    )
    self.detail.delete("1.0", "end")
    self.detail.insert("1.0", text)

def add_custom_case_window(self):
    win = tk.Toplevel(self)
    win.title("Add Custom Repair Case")
    win.geometry("520x520")
    win.configure(bg="#f4f7fb")

    fields = {}
    labels = [
        ("device", "Device (laptop/phone)",),
        ("fault", "Fault name",),
        ("symptoms", "Symptoms, comma separated atoms",),
        ("severity", "Severity (low/medium/high/critical)",),
        ("cost", "Cost (low/medium/high)",),
        ("backup", "Backup needed (yes/no)",),
        ("advice", "Repair advice",),
    ]
    for key, label in labels:
        anchor="w", padx=18, pady=(10, 2))
        entry = ttk.Entry(win, width=70)
        entry.pack(fill="x", padx=18)
        fields[key] = entry

    fields["device"].insert(0, self.device_var.get())
    fields["severity"].insert(0, "medium")
    fields["cost"].insert(0, "medium")
    fields["backup"].insert(0, "no")

def save_case():
    device = safe_atom(fields["device"].get())
    fault = safe_atom(fields["fault"].get())
    symptoms = [safe_atom(s) for s in fields["symptoms"].get().split(",") if s.strip()]
    severity = safe_atom(fields["severity"].get())

    cost = safe_atom(fields["cost"].get())
    backup = safe_atom(fields["backup"].get())
    advice = fields["advice"].get().replace("'", "")
    label = fields["fault"].get().replace("'", "").strip() or fault
    if not symptoms:
        messagebox.showwarning("Missing symptoms", "Add at least one symptom atom.")
        return
    with open(CUSTOM_FILE, "a", encoding="utf-8") as f:
        f.write(f'\nfault_info({fault}, {device}, "{label}", {severity}, {cost}, "{advice}", {
backup}).\n')
        f.write(f'\nfault_symptoms({device}, {fault}, {prolog_list(symptoms)}).\n')
    messagebox.showinfo("Saved", "Custom case saved to custom_cases.pl. Restart app or diagnose
again.")
    win.destroy()

    ttk.Button(win, text="Save Custom Case", command=save_case).pack(pady=20)

def open_folder_hint(self):
    messagebox.showinfo("Project Folder", f"Project folder:\n{BASE_DIR}")

if __name__ == "__main__":
    app = RepairDiagnosisApp()
    app.mainloop()

```

## 18. References

- CM2520 Deductive Reasoning and Logic Programming lecture notes uploaded in the conversation.
- DRLP Project Guidelines PDF uploaded in the conversation.
- SWI-Prolog documentation for predicates such as assertz/1, retractall/1, findall/3, member/2 and dynamic/1.
- Python Tkinter standard library documentation for GUI development.

Note: The repair advice in this project is for educational demonstration. Real repair decisions should be confirmed by a qualified technician, especially for water damage, battery swelling, data recovery and motherboard-level faults.